ECE521- MICROPROCESSOR SYSTEM

MINI PROJECT

CHILDREN FALLING PREVENTION SYSTEM IN HIGH-RISE BUILDING

(CEEE2224B)

PREPARED BY:

NAME	STUDENT ID
ANIS SUHANA BINTI MOHD AZMI	2023816852
MAISARAH BINTI LISMAHDI	2023800492
ALISSYA BINTI ZAIM	2023867864

The group lab report was presented to Madam Farah Yasmin Abdul Rahman as part of the subject requirements.

ECE521- MICROPROCESSOR SYSTEM

FACULTY OF ELECTRICAL ENGINEERING

BACHELOR OF ELECTRONIC ENGINEERING WITH HONOURS

SESI 2023/2024

TABLE OF CONTENTn

CHAPTER	TITLE	PAGE
1	**INTRODUCTION** 1.1 Introduction 1.2 Problem Statement 1.3 Project Objectives	2-5
2	**HARDWARE DESIGN** 2.1 Block Diagram 2.2 Circuit Design	6-9
3	**SOFTWARE DESIGN** 3.1 Flowchart 3.2 Coding of the Project 3.3 Bluetooth application design using MIT Apps Inventor	10-27
4	**RESULT AND DISCUSSION** 4.1 Prototype 4.2 Prototype Operation 4.3 Test and Troubleshoot	28-40
5	**CONCLUSION** 5.1 Conclusion	41-44
6	**REFERENCE** 6.1 Reference	45

CHAPTER 1

INTRODUCTION

TITLE: CHILDREN FALLING PREVENTION SYSTEM IN HIGH-RISE BUILDING

1.1 Introduction

Urbanisation is one of the main reasons pushing the construction of high-rise and vertical industrial growth in developed countries, especially in areas with high population density, in order to ensure appropriate housing availability. The ongoing expansion of high rise housing developments highlights the critical need for an effective property management system to promote environmentally friendly growth in these urban communities. Children living in high-rise residential buildings are especially exposed to hazards, particularly the risk of falling from high places, which has been a global issue for many years, frequently resulting in life-threatening injuries and fatalities. While incorporating simple child safety designs for newly constructed homes or renovating existing homes with child safety elements can help reduce these potential hazards, effective implementation requires policy and law enforcement intervention.

This project aims to solve these safety concerns by designing, developing, and prototyping a children's fall prevention system using STM32F446RE microcontroller. The proposed system uses a variety of sensors and actuators to identify possible fall risks and quickly inform caretakers, allowing for immediate action. The reason for this project comes from the increasing urbanisation and the rise in high-rise residential buildings, making the security of children a critical concern for parents and caregivers. Typical security measures, such as window guards and balcony nets, provide a level of protection but they are not sufficient or practical in all scenarios. By utilising advanced microcontroller technology and a variety of sensors, this project aims to develop a reliable, cost-effective solution to prevent tragic accidents and improve children safety in high-rise situations.

High-rise buildings, while beneficial for urban environments with limited space, they also offer considerable safety challenges. One of the most alarming risks is the possibility of falling

from balconies, windows, or other elevated areas, which can lead to severe injuries or fatalities. Developing a comprehensive children fall prevention system is crucial to address these safety concerns and protect young lives. With urbanisation on the rise, families are increasingly living in multi-story apartments, increasing the risk of unintentional falls for children. Traditional safety precautions may not always be effective or practical, forcing the development of more innovative and automated alternatives.

By utilising today's technology, one can create an effective, automated solution that actively monitors and responds to potential fall hazards, giving parents and caregivers peace of mind. This project proposes a system that uses the STM32F446RE microcontroller to integrate various sensors and actuators. Ultrasonic sensors estimate the distance to potential obstacles, infrared sensors detect movement in specific areas, and a Bluetooth module allows for real time communication with caregivers' smartphones. The system also includes visual and auditory alarms, such as LEDs, a buzzer, and a push button, as well as a dc motor for physical assistance, such as closing a window or triggering a safety barrier.

The implementation of such a system involves several key steps, including hardware design, software development, prototyping, and testing. Additionally, one can build a functional prototype by assembling the hardware components and implementing the software that manages sensor inputs and controls actuators. Thorough testing and troubleshooting ensure that the system operates reliably under various conditions, resulting in an effective solution for the safety of children in high-rise structures.

Subsequently, this project not only addresses a critical safety need but also demonstrates the potential of modern technology to create smarter, safer living environments. The project's goal is to create a dependable, effective solution that can be used in high-rise structures all around the world through attentive design, development and testing. This will improve protection and provide families and caregivers peace of mind, ensuring that the children may live safely in high-rise buildings. Moreover, the initiative emphasises the role of policy and regulation in enforcing child safety measures, pushing for the implementation of these features into both new and existing high-rise residential properties. The study's findings aim to ensure the implementation of child safety policies to prevent child falls and make high-rise buildings safer for children to live.

1.2 Problem Statement

With the increasing trend of modernisation and the subsequent development in high-rise residential buildings, there is a significant risk of falls from heights, particularly among children. Current safety precautions, such as window guards and balcony nets, may not be appropriate or practical in all situations, creating persistent risk that must be addressed with more reliable and automated solutions.

Existing child safety designs and features frequently lack integration and automation, making them less effective in preventing falls from high-rise buildings. There is a need for a comprehensive system capable of actively monitoring possible fall hazards and providing timely alarms and interventions to avoid accidents.

While simple child safety designs can help prevent accidents, their execution is often inconsistent due to lack of awareness and enforcement of safety rules and regulations. This inconsistency emphasises the necessity for a standardised, technologically advanced solution that can be broadly adopted and required by policy and legislation.

Renovating existing high-rise residential buildings with children safety features is both practical and financially challenging. A cost-effective, easy to install solution is required to ensure that all high-rise buildings may be made child-safe without requiring considerable modifications or spending significant costs.

Traditional child safety measures rely heavily on constant supervision from parents or caregivers, which is not always possible. There is a need for an automated system that can provide real-time monitoring and alerts, minimising the caregivers' workload and ensuring child safety even when they are not there.

Establishing a user-friendly system that incorporates a variety of sensors, microcontrollers, and communication components can be challenging. The effective implementation of a child prevention system in high-rise buildings depends on ensuring seamless operation, convenience of use, and reliable performance.

1.3 PROJECT OBJECTIVES

1. To design hardware and software for children fall prevention systems in high-rise buildings using STM32F445RE microcontroller.

2. To build a prototype of a Children Fall Prevention System in a high-rise building.

3. To test and troubleshoot the prototype of project Children Falling Prevention system.

CHAPTER 2

HARDWARE DESIGN

2.1 Block Diagram

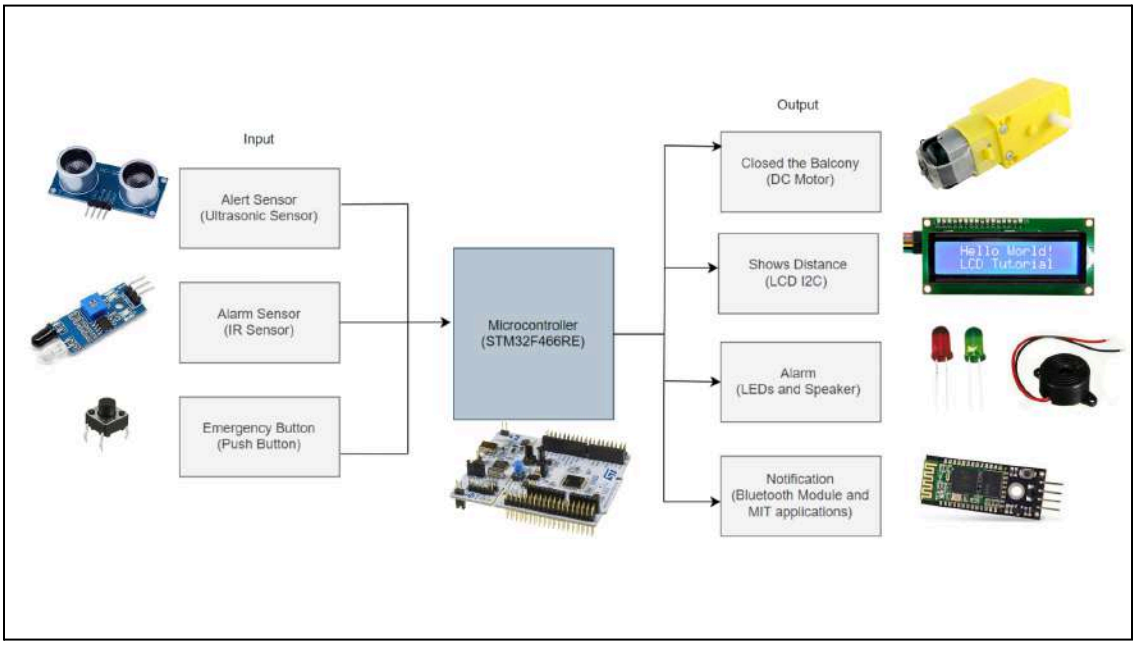


Figure 1.1: block diagram of the project

Figure 1.1 shows the block diagram of the project. For the input of the project, the ultrasonic sensor acts as an alert sensor. When the ultrasonic sensor detects an object, its value will be displayed on the LCD I2C. Next, the IR sensor acts as an alarm sensor. When the IR sensor detects something, the red LED will turn on and the alarm speaker will also activate. Finally, the push button acts as an emergency button. When the push button is pressed, the servo motor will turn 180 degrees to close the balcony immediately, ensuring the child's safety and preventing a fall.

2.2 Circuit Design

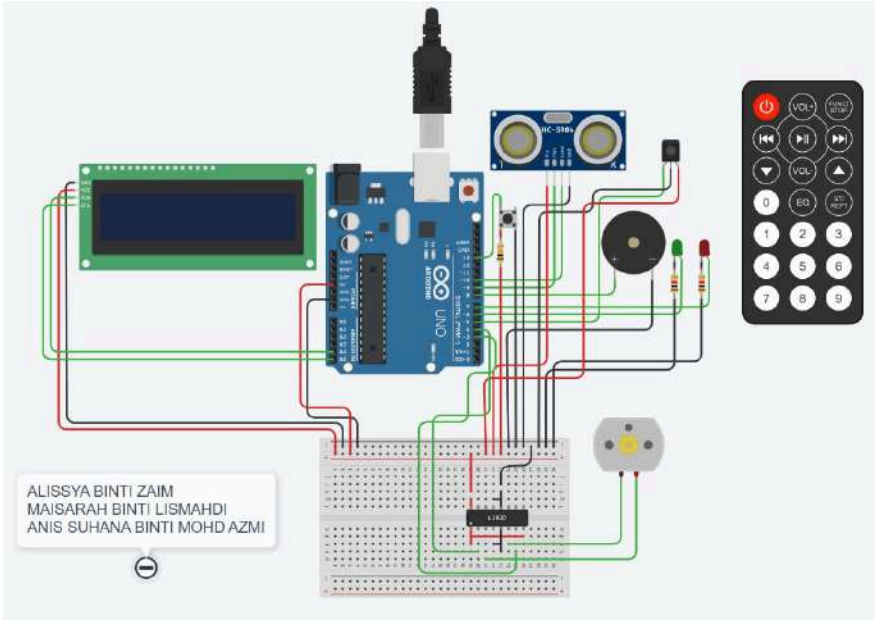


Figure 2.2: Circuit Design

For the circuit design, Arduino Uno is being used to run this project since Tinkercad Software is lacking of STM32F446RE board, an LCD display connected via I2C (SDA to A4, SCL to A5), an HC-SR04 ultrasonic sensor (Trig to Digital Pin 9, Echo to Digital Pin 10), and an IR sensor (output to Digital Pin 5). A DC motor controlled by an L293D motor driver (IN1 to Digital Pin 3, IN2 to Digital Pin 4) is used to open and close a door. The motor is powered by an external supply, while the logic supply is connected to the Arduino's 5V. A buzzer connected to Digital Pin 8 emits sound when the IR sensor detects an object. A push button connected to Digital Pin 13 (in a pull-down configuration) allows for manual control, causing the motor to rotate 180 degrees anticlockwise. The red LED (anode to Digital Pin 7) and green LED (anode to Digital Pin 6) indicate the system status based on the IR sensor's detection. The HC-SR04 sensor's distance measurements are displayed on the LCD, and the entire setup is powered and managed by the Arduino, ensuring the motor rotates only when specific conditions are met, with LEDs and the buzzer indicating status changes.

2.3 Schematics Diagram

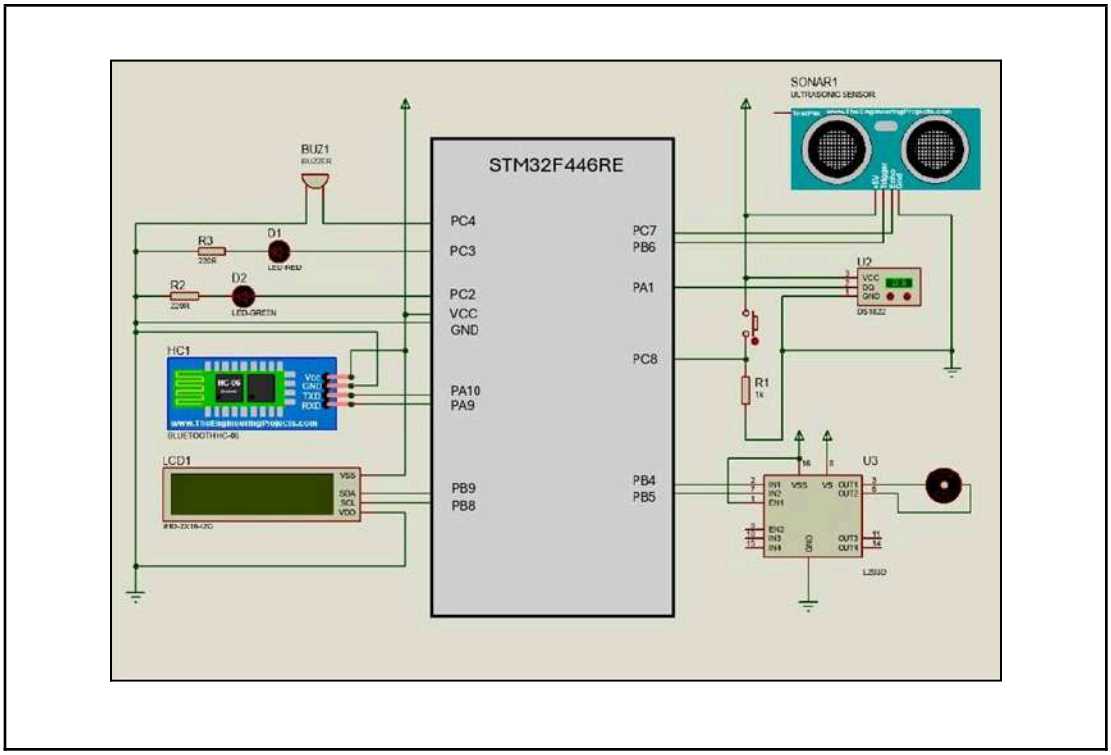


Figure 2.3: circuit diagram

Figure 2.3 shows the circuit diagram of this project. In the project circuit diagram, the buzzer is connected to pin PC4, as an alert system. A red LED is connected to PC3 to indicate warning, while a green LED at PC2 shows normal operation. The Bluetooth module, which enables wireless communication with other devices, is connected via PA10 (RX) and PA9 (TX). For display purposes, an LCD I2C module is connected to PB9 (SDA) and PB8 (SCL), with pull-up resistors ensuring proper communication.

The ultrasonic sensor, used for measuring distances and obstacle detection, connects its trigger to PC7 and echo to PB6. An IR sensor, which detects infrared light for object or proximity sensing, is connected to PA1. A push button is connected to the PCB, allowing manual input to trigger specific actions within the system. The DC motor, controlled through a motor driver, is linked to PB4 and PB5 of the STM32F466RE, facilitating precise control over its speed and direction.

Ensuring a stable power supply to the STM32F466RE and all components is crucial, typically using a 5V regulated power source. The microcontroller acts as the central unit,

processing sensor inputs, managing outputs like LEDs and the buzzer, and handling communication through Bluetooth and the LCD I2C module. Each component connection includes appropriate resistors, level shifters, or pull-up resistors as needed to maintain voltage levels and ensure proper functioning.

By organising these connections on a breadboard and donut board, the circuit remains secure and efficient, allowing for easy troubleshooting. This setup ensures that all components work well within the project, providing accurate sensing, communication, and control capabilities.

CHAPTER 3

SOFTWARE DESIGN

3.1 Flowchart

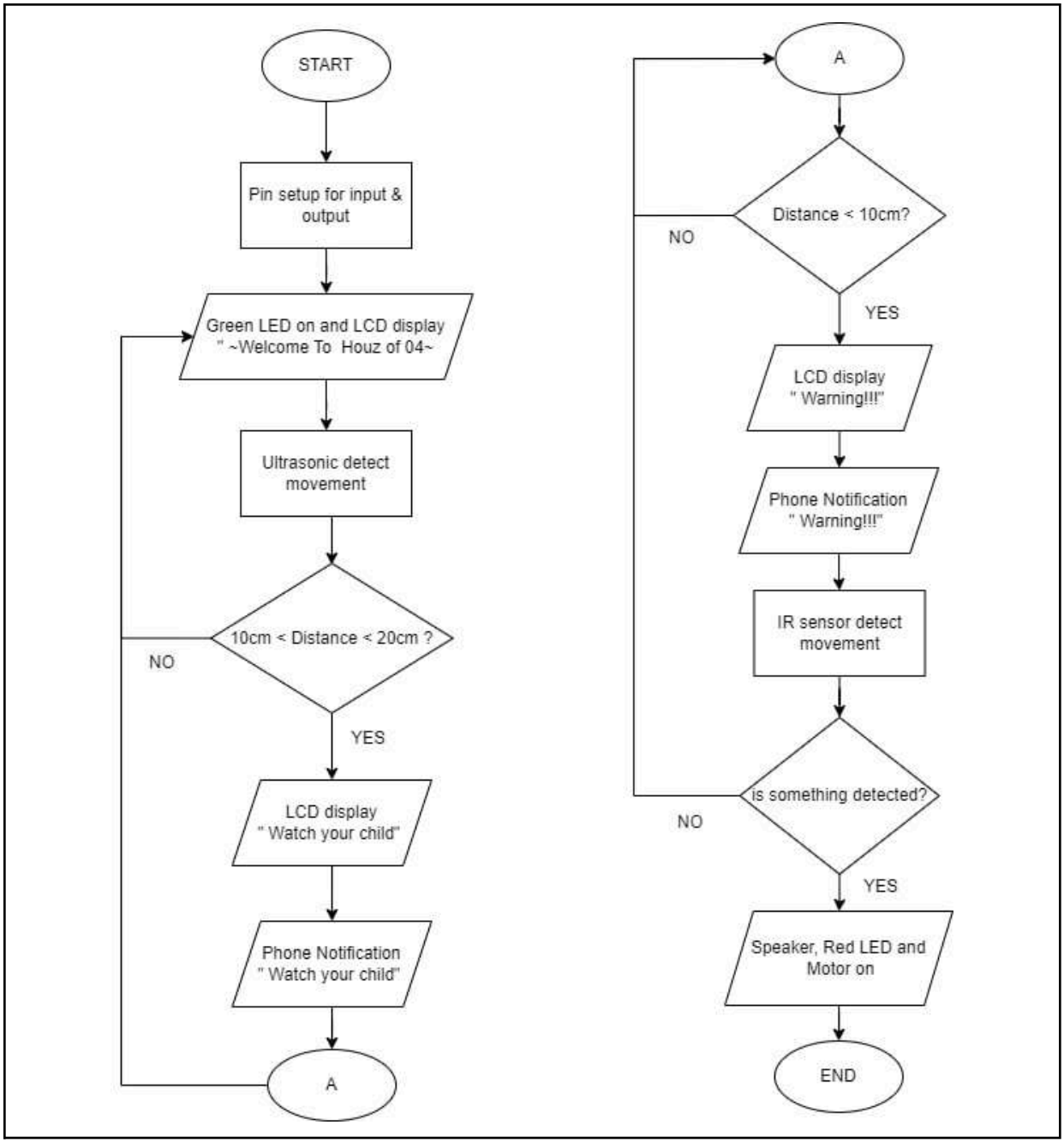


Figure 3.1.1 : The Flowchart of The Project

The children fall prevention system begins with the initialization of the hardware components, including the STM32F446RE microcontroller, ultrasonic sensors, infrared (IR) sensors,buzzer, LCD (I2C) display, LEDs, push button, and Bluetooth module. Once the system is powered on, it enters a standby mode where it continuously monitors the environment for potential hazards.

The system features an ultrasonic sensor to determine the distance between the child and the exterior of a high-rise building. The sensor sends out ultrasonic waves and measures the time taken for the waves to bounce back after hitting an object. The distance is calculated based on the time of flight of the waves. If the measured distance is greater than 10 cm and lesser than 20 cm, the LCD displays a “watch your child” message, and the system will send notification to the parents or caregivers smartphone via the Bluetooth module. If the distance is less than or equal to 10 cm, the system enters a critical state. It displays a warning message on the LCD, and sends a notification to the caregiver's smartphone via the Bluetooth module.

In parallel, the system uses an IR sensor to detect movement at a specific point, such as near a door or balcony. When the IR sensor detects movement, the system considers it as a potential fall risk. It triggers the red LED to turn on, indicating a high-risk zone, and rotates the dc motor 180 degrees to close a protective barrier, such as a door or window. The buzzer is also activated to alert nearby individuals.

The system includes a manual override feature using a push button. If the button is pressed, the system checks the current position of the dc motor. If the motor is in the closed position, it rotates the servo motor 180 degrees anti-clockwise to open the protective barrier. This manual control allows caregivers to override the automatic safety features when necessary.

Throughout the operation, the system continuously monitors the sensors and updates the status on the LCD and the Bluetooth module. It ensures real-time monitoring and prompt alerts to address potential fall risks. By integrating these features, the children fall prevention system provides a comprehensive solution to monitor and prevent fall risks in high-rise buildings. The use of sensors, microcontroller technology, and real-time alerts ensures a reliable and effective safety mechanism to protect children from accidental falls.

3.2 Coding

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>

const int irPin = PA1;      // IR sensor
const int ledPinG = PC2;    // LED GREEN
const int ledPinR = PC3;    // LED RED
const int buttonPin = PC8;  // Button
const int motorPin1 = PB4;  // DC Motor control pin 1
const int motorPin2 = PB5;  // DC Motor control pin 2
const int speakerPin = PC4; // Buzzer
const int trigPin = PC7;    // Ultrasonic sensor Trig
const int echoPin = PB6;    // Ultrasonic sensor Echo

LiquidCrystal_I2C lcd(0x27, 16, 2);
SoftwareSerial hc05(PA10, PA9);

bool lastButtonState = HIGH;
bool currentButtonState;
bool motorAtInitialPosition = true;
bool objectDetected = false;

void setup() {
  Serial.begin(9600);
  hc05.begin(9600);

  pinMode(irPin, INPUT);
  pinMode(ledPinG, OUTPUT);
  pinMode(ledPinR, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
```

```
  pinMode(speakerPin, OUTPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(motorPin1, OUTPUT);
  pinMode(motorPin2, OUTPUT);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Houz system ON..");
  delay(1500);
  lcd.clear();
}
void loop() {


  long distance = getDistance();

  if (distance <= 10) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Warning!!!");
    lcd.setCursor(0, 1);
    lcd.print("Distance: ");
    lcd.print(distance);
    lcd.print(" cm");

    hc05.println("Warning!!!");
    Serial.println("Warning!!!");

  } else if (distance <= 20) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("WATCH YOUR CHILD");
```

```
    lcd.setCursor(0, 1);
    lcd.print("Distance: ");
    lcd.print(distance);
    lcd.print(" cm");

    hc05.println("WATCH YOUR CHILD");
    Serial.println("WATCH YOUR CHILD");

  } else {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("  ~Welcome To ");
    lcd.setCursor(0, 1);
    lcd.print("   Houz of 04~");
  }
  delay(100);
}
int getDistance() {
  long duration, distance;
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  distance = duration * 0.034 / 2;
  return distance;
}
void rotateMotorClockwise(int degrees) {
  digitalWrite(motorPin1, HIGH);
  digitalWrite(motorPin2, LOW);
  delay(calculateDelay(degrees));
  stopMotor();
```

```
}
void rotateMotorAnticlockwise(int degrees) {
  digitalWrite(motorPin1, LOW);
  digitalWrite(motorPin2, HIGH);
  delay(calculateDelay(degrees));
  stopMotor();
}
void stopMotor() {
  digitalWrite(motorPin1, LOW);
  digitalWrite(motorPin2, LOW);
}
int calculateDelay(int degrees) {
 for the given degrees
  const int delayPerDegree = 10;
  return degrees * delayPerDegree;
}
```

Figure 3.2.1 : The Full Coding of The Project

This code in Figure 3.2.1 is created with various combinations of code components connected to the STM32F446RE board, including input and output components such as IR sensors, ultrasonic sensors and buttons for input. While the output is LED, DC motor, buzzer, LCD I2C and Bluetooth module. The setup function configures the pin mode and initialises the LCD. In the loop function, it checks for the presence of an object using an IR sensor, and will set the LED and buzzer accordingly, and rotate the motor to a certain position. It also reads the state of the button to reset the motor position after the motor rotates when the IR sensor detects. Next, it uses an ultrasonic sensor to measure the distance for 20 cm to 15cm and a distance less than 15 cm, and updates the LCD display and sends a Bluetooth notification if a certain distance condition is met. The function getDistance calculates the distance using the ultrasonic sensor, while the functions of rotateMotorClockwise and rotateMotorAnticlockwise control the direction and position of the motor. For further explanation related to this coding, it will be explained in detail below about the function and use of what code is used.

```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <SoftwareSerial.h>
```

Figure 3.2.2 : The library in the program.

In Figure 3.2.2 there are three lines of code including three libraries that are important for different functions in the program. The first line is `#include <Wire.h>` which adds the Wire library, and is used for I2C communication to allow the microcontroller to communicate with other devices. The second line which is `#include <LiquidCrystal_I2C.h>` is the LiquidCrystal_I2C library, which enables LCD screen control via I2C. The last line, `#include <SoftwareSerial.h>` which includes the SoftwareSerial library, it allows the creation of a serial communication port using digital pins, useful for connecting the Bluetooth module in this project.

```
5  // Pin definitions for STM32 Nucleo-446RE
6  const int irPin = PA1;       // IR sensor
7  const int ledPinG = PC2;     // LED GREEN
8  const int ledPinR = PC3;     // LED RED
9  const int buttonPin = PC8;   // Button
10 const int motorPin1 = PB4;   // DC Motor control pin 1
11 const int motorPin2 = PB5;   // DC Motor control pin 2
12 const int speakerPin = PC4; // Buzzer
13 const int trigPin = PC7;     // Ultrasonic sensor Trig pin
14 const int echoPin = PB6;     // Ultrasonic sensor Echo pin
```

Figure 3.2.3 : The Definition of Pin in The Program.

In this figure, this code specifies the pins on the STM32 Nucleo-446RE board that are connected to the various components in the project. That is `const int irPin = PA1;` pin for IR sensor, `const int ledPinG = PC2;` pin for green LED, `const int ledPinR = PC3;` pin for red LED, `Const int buttonPin = PC8;` pin for button, `const int motorPin1 = PB4;` and `const int motorPin2 = PB5;` to control two DC motor control pins, `Const int speakerPin = PC4;` pins for speakers, `const int trigPin = PC7;` and `const int echoPin = PB6;` set for ultrasonic sensor trigger and echo pin respectively. Pin definition is easier and clearer to see, in coding it is very important before starting main system code.

```
16 // LCD I2C setup
17 LiquidCrystal_I2C lcd(0x27, 16, 2); // Address 0x27, 16 columns, 2 rows
18
19 // Bluetooth setup
20 SoftwareSerial hc05(PA10, PA9); // RX, TX (use pins for software serial)
21
22 // Button part
23 bool lastButtonState = HIGH;
24 bool currentButtonState;
25 bool motorAtInitialPosition = true;
26 bool objectDetected = false;
```

Figure 3.2.4 : The Setup of device in The Program.

In the figure 3.2.4 there is this line of code providing the LCD display, Bluetooth module and some button state variables and motor position. For line `LiquidCrystal_I2C lcd (0x27, 16, 2);` it initialises the LCD display with I2C address `0x27`, and has 16 columns and 2 rows. For line `SoftwareSerial hc05 (PA10, PA9);` is the Bluetooth module on pins PA10 (RX) and PA9 (TX). For buttons and motors, `bool lastButtonState = HIGH;` and `bool currentButtonState;` are functions to track the previous and current state of the button. `bool motorAtInitialPosition = true;` to know if the motor is in the initial position and `bool objectDetected = false;` to track if an object has been detected.

```
28 void setup() {
29   Serial.begin(9600);       // Debugging serial communication
30   hc05.begin(9600);         // Bluetooth serial communication
31
32   // Pin modes setup
33   pinMode(irPin, INPUT);
34   pinMode(ledPinG, OUTPUT);
35   pinMode(ledPinR, OUTPUT);
36   pinMode(buttonPin, INPUT_PULLUP);
37   pinMode(speakerPin, OUTPUT);
38   pinMode(trigPin, OUTPUT);
39   pinMode(echoPin, INPUT);
40   pinMode(motorPin1, OUTPUT);
41   pinMode(motorPin2, OUTPUT);
42
43   // LCD initialization
44   lcd.init();
45   lcd.backlight(); // Trun on LCD backlight
46   lcd.setCursor(0, 0); // Set cursor to column 0, row 0
47   lcd.print("Houz system ON..");
48   delay(1500); // Wait for 1.5 seconds
49   lcd.clear();
50 }
```

Figure 3.2.5 : The Void Setup in The Program.

In the figure 3.2.5 this first line of code is the initial configuration for serial communication. `Serial.begin(9600);` starts serial communication at 9600 baud rate for debugging, and `hc05.begin(9600);` starts serial communication for Bluetooth modules at the same rate. The `pinMode` function configures the specified pins as : `irPin` as input, `ledPinG` and `ledPinR` as output, `buttonPin` as input, `speakerPin` as output, `trigPin` as output, `echoPin` as input and `motorPin1` and `motorPin2` as outputs. Next for the LCD, `lcd.init();` to initialise the display, `lcd.backlight();` to turn on the backlight, `lcd.setCursor(0, 0);` to place the cursor in the upper left corner, and ` lcd.print("Houz system ON..");` to display the message "Houz system ON..". After a 1.5 second delay (`delay(1500);`), clear the display using `lcd.clear();`.

```
void loop() {
  // Check IR sensor state
  if (digitalRead(irPin) == LOW && !objectDetected) {
    // IR sensor detects object
    objectDetected = true;
    digitalWrite(ledPinG, LOW);  // Turn off green LED
    digitalWrite(ledPinR, HIGH); // Turn on red LED
    tone(speakerPin, 1000);      // Turn on speaker

    // Operate DC motor clockwise if at initial position
    if (motorAtInitialPosition) {
      rotateMotorClockwise(180);   // Rotate 180 degrees clockwise
      motorAtInitialPosition = false;
    }

    Serial.println("IR detected: Door closed");
  } else if (digitalRead(irPin) == HIGH) {
    // IR sensor does not detect object
    objectDetected = false;
    digitalWrite(ledPinG, HIGH); // Turn on green LED
    digitalWrite(ledPinR, LOW);  // Turn off red LED
    noTone(speakerPin);          // Turn off speaker
    }

  // Check button state to control motor
  currentButtonState = digitalRead(buttonPin);
  if (currentButtonState == LOW && lastButtonState == HIGH && !motorAtInitialPosition) {
    // Button pressed, reverse motor direction
    rotateMotorAnticlockwise(180); // Rotate 180 degrees anticlockwise
    motorAtInitialPosition = true;
    Serial.println("Button pressed: Door opened");
  }
  lastButtonState = currentButtonState; // Update last button state
```

Figure 3.2.6 : The Void Loop Part 1 in The Program.

In figure 3.2.6 there is code in the `loop()` function that runs continuously until the system is off. The IR sensor here at the designated point will signal that the red LED, speaker and Motor (clockwise) are active. Initially, the IR sensor detects an object (`digitalRead(irPin) == LOW`) and if no object was detected before (`!objectDetected`). If both conditions are true, it sets `objectDetected` to true, turns off the green LED (`digitalWrite(ledPinG, LOW)`), turns on the red LED (`digitalWrite(ledPinR, HIGH)`), and activates the speaker (`tone(speakerPin, 1000)`). If the motor is in its initial position (`motorAtInitialPosition`), it rotates the motor clockwise by 180 degrees (`rotateMotorClockwise(180)`) and sets `motorAtInitialPosition` to false. Next, print the message to the serial monitor in the Arduino IDE. If the IR sensor does not detect an object (`digitalRead(irPin) == HIGH`), it sets `objectDetected` to false, turns on the green LED (`digitalWrite(ledPinG, HIGH)`), turns off the red LED (`digitalWrite(ledPinR,LOW)`),and turn off the speaker (`noTone(speakerPin)`).

Next, on the push button, when pressed it will rotate the motor anti-clockwise to show the balcony slide door is open. It reads the state of the button and checks if the button is pressed (`currentButtonState == LOW && lastButtonState == HIGH`) and if the motor is not in its initial position (`!motorAtInitialPosition`). If true, it rotates the motor counterclockwise by 180 degrees (`rotateMotorAnticlockwise(180)`) and sets `motorAtInitialPosition` to true, then prints a message to the serial monitor. Finally, it updates `lastButtonState` to the current button state.

```
  // Measure distance using ultrasonic sensor
  long distance = getDistance(); // get distance

  // Update LCD and send Bluetooth notifications
  if (distance <= 10) {
    lcd.clear();
    lcd.setCursor(0, 0); // Set cursor to the first column and first row
    lcd.print("Warning!!!");
    lcd.setCursor(0, 1); // Set cursor to the second line
    lcd.print("Distance: ");
    lcd.print(distance); // Display distance on LCD
    lcd.print(" cm");

    // Send Bluetooth notification
    hc05.print("Warning!!!");
    Serial.println("Warning!!!");


  } else if (distance <= 20) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("WATCH YOUR CHILD");
    lcd.setCursor(0, 1);
    lcd.print("Distance: ");
    lcd.print(distance);
    lcd.print(" cm");

    // Send Bluetooth notification
    hc05.print("WATCH YOUR CHILD");
    Serial.println("WATCH YOUR CHILD");

  } else {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("  ~Welcome To ");
    lcd.setCursor(0, 1);
    lcd.print("   Houz of 04~");
  }

  delay(100); // Short delay for stability
}
```

Figure 3.2.7 : The Void Loop Part 2 in The Program.

In this figure 3.2.7, the code used to measure the distance using an ultrasonic sensor, displays an LCD display and a Bluetooth notification based on the distance. The first line, `distance = getDistance();` calls the function to measure the distance. If the distance is less than or equal to 10 cm, the LCD is cleared (`lcd.clear()`), the cursor is set to the first row and column (`lcd.setCursor(0, 0)`), and the message "Warning!!!" displayed (`lcd.print("Warning!!!")`). The cursor is then moved to the second line (`lcd.setCursor(0, 1)`) and the distance is displayed with "Distance:" followed by the distance in cm (`lcd.print(distance); lcd.print( "cm")`). Bluetooth notification "Warning!!!" sent using `hc05.print("Warning!!!");`and printed to the serial monitor with `Serial.println("Warning!!!");. Whereas, if the distance is more than 10 cm but less than or equal to 20 cm, the same process takes place but with the message "WATCH YOUR CHILD". In addition, If the distance condition is other than what is set in the code, the LCD is blank and the message "Welcome To Houz of 04" is displayed on the LCD. Finally, there is a short delay (`delay(100);`) to keep the system stable.

```
128 int getDistance() {
129   long duration, distance;     // Variables to store duratian and distance
130   digitalWrite(trigPin, LOW);  // Clear trigger
131   delayMicroseconds(2);        // Set the trig pin to LOW for 2 microseconds
132   digitalWrite(trigPin, HIGH); // Send Trigger pulse
133   delayMicroseconds(10);       // Wait for 10 microseconds
134   digitalWrite(trigPin, LOW);  // Stop pluse
135
136   duration = pulseIn(echoPin, HIGH); // Get the pulse duration in microseconds
137   distance = duration * 0.034 / 2;   // Calculation convert to centimeters
138
139   return distance; // Return the measured distance
140 }
141 void rotateMotorClockwise(int degrees) { // Rotate motor clockwise
142   digitalWrite(motorPin1, HIGH);
143   digitalWrite(motorPin2, LOW);
144   delay(calculateDelay(degrees));        // Delay based on motor speed
145   stopMotor();                           // Stop the motor after rotation
146 }
147 void rotateMotorAnticlockwise(int degrees) { // Rotate motor anti-clockwise
148   digitalWrite(motorPin1, LOW);
149   digitalWrite(motorPin2, HIGH);
150   delay(calculateDelay(degrees));           // Delay based on motor speed
151   stopMotor();                              // Stop the motor after rotation
152 }
153 void stopMotor() {                         // Function for stop motor
154   digitalWrite(motorPin1, LOW);
155   digitalWrite(motorPin2, LOW);
156 }
157 int calculateDelay(int degrees) {
158   const int delayPerDegree = 10; // Adjust this value based on the motor speed and gear ratio
159   return degrees * delayPerDegree; // Return the calculated delay in miliseconds
160 }
```

Figure 3.2.8 : The Functions in The Program.

From the Figure 3.2.8 has 4 functions that use which the `getDistance` function measures the distance using an ultrasonic sensor. The code starts by defining two variables, `duration` and `distance`, to store the calculated duration and distance. Then set `trigPin` to LOW for 2 microseconds and then to HIGH for 10 microseconds to send a trigger pulse, which then stops by setting `trigPin` back to LOW. The `pulseIn` function measures the time it takes for the echo to return to the `echoPin`. And then converted to distance in centimetres using the formula `distance = period * 0.034 / 2`, and the function returns this distance. Then for the `rotateMotorClockwise` function rotates the motor clockwise by the specified number of degrees. It sets `motorPin1` to HIGH and `motorPin2` to LOW to start the rotation, a delay calculated based on the degrees specified (`delay(calculateDelay(degrees))`), and then there is a `stopMotor` function to stop. the motor. Likewise, the `rotateMotorAnticlockwise` function rotates the motor counterclockwise. It sets `motorPin1` to LOW and `motorPin2` to HIGH, a delay is calculated, and then there is a `stopMotor` function to stop the motor. The `stopMotor` function used in the `rotateMotorClockwise` and `rotateMotorAnticlockwise` functions is to stop the motor by setting both `motorPin1` and `motorPin2` to LOW.

Finally, the `calculateDelay` function calculates the delay required to rotate the motor according to the specified number of degrees. It multiplies the degrees by `delayPerDegree`, which represents the delay per degree based on the speed and gear ratio of the motor, and then returns the calculated delay in milliseconds. In short, the use of these functions can make the code more efficient, structured, and easy to maintain.

3.3 Bluetooth Application Design using MIT App Inventor

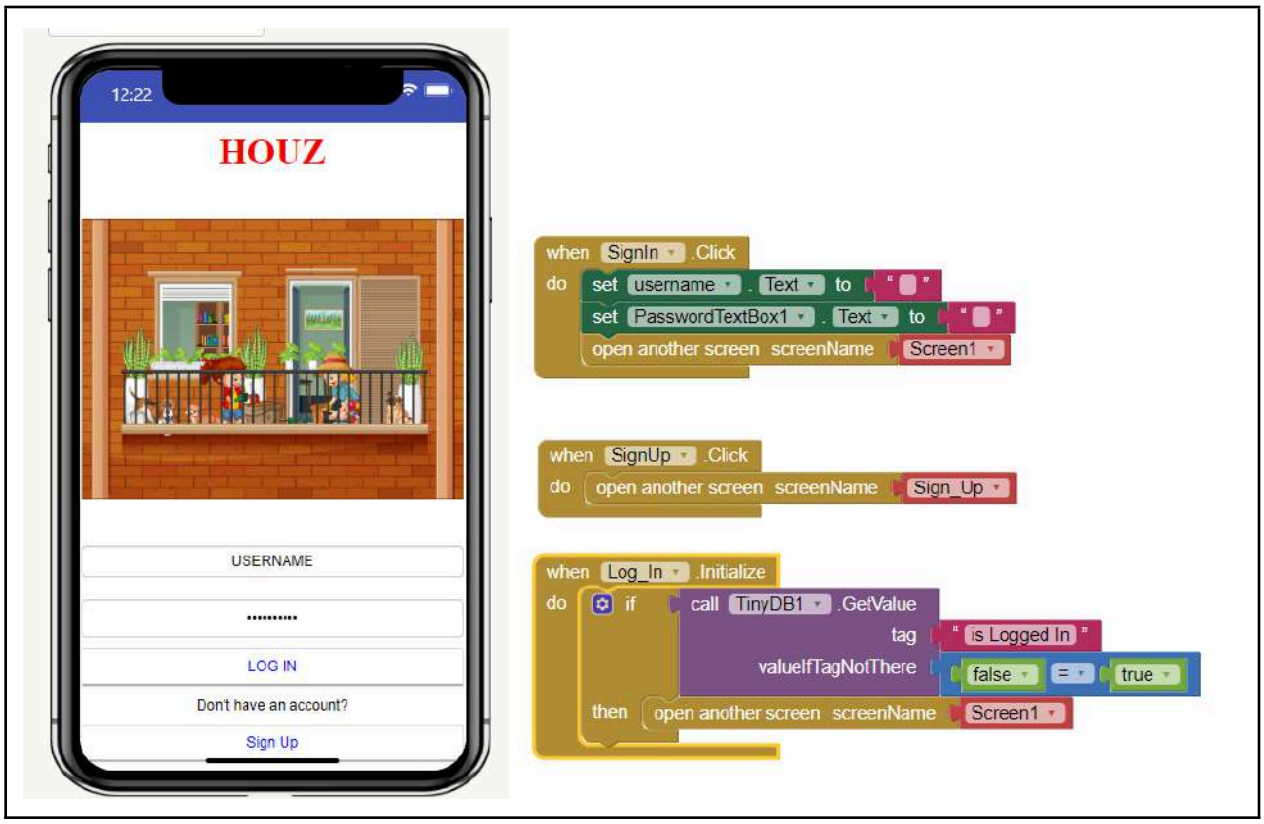


Figure 3.3.1: LogIn Interface on MIT App Inventor

The login page is a crucial component of the mobile application for the children's fall prevention system, since it allows caregivers to access and control the system. The login page is designed to be simple and easy to use, allowing caregivers to access the application quickly and securely. The login page is designed to be visually appealing and easy to navigate. The elements are arranged in a vertical layout, with ample spacing between them to prevent any clutter. The username and password fields are placed near the top, followed by the login button, and the sign up button at the bottom. By incorporating these elements and functionality, the login page ensures that only authorised users can access the children fall prevention system. This security measure helps protect the system from unauthorised access and ensures that caregivers can effectively monitor and respond to potential fall risks. The design and functionality of the login page aim to provide a seamless and secure user experience.

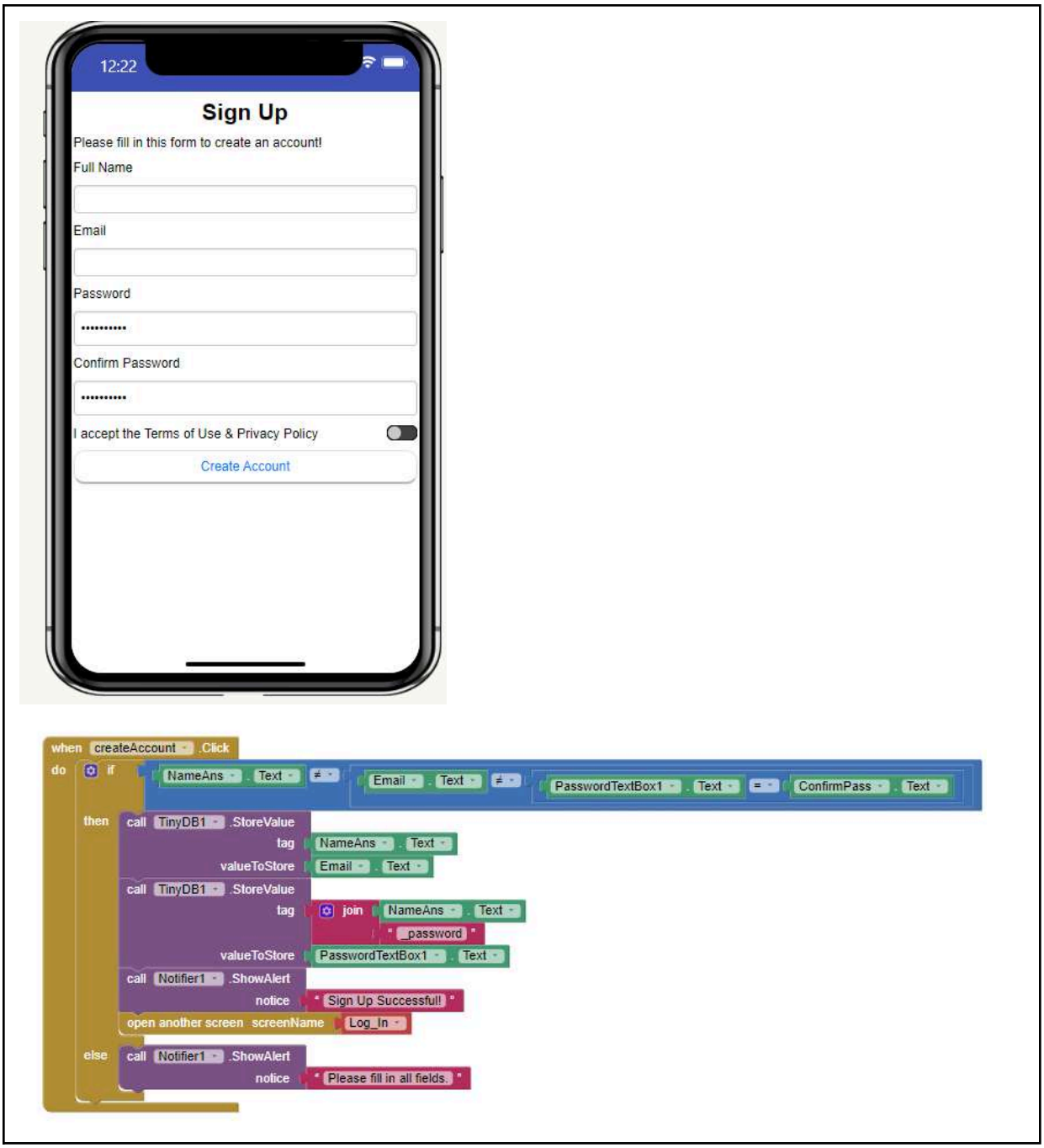


Figure 3.3.2: Sign Up Interface on MIT App Inventor

The sign-up page is designed to allow new users to create an account in the children fall prevention system app. This page will collect basic information from the users, such as their name, email, and password, and store this information securely. The sign-up process ensures that only authorised users can access the app's features and receive notifications from the system.This sign-up page ensures that only registered users can access the

children fall prevention system app, enhancing the security and personalization of the app's features.

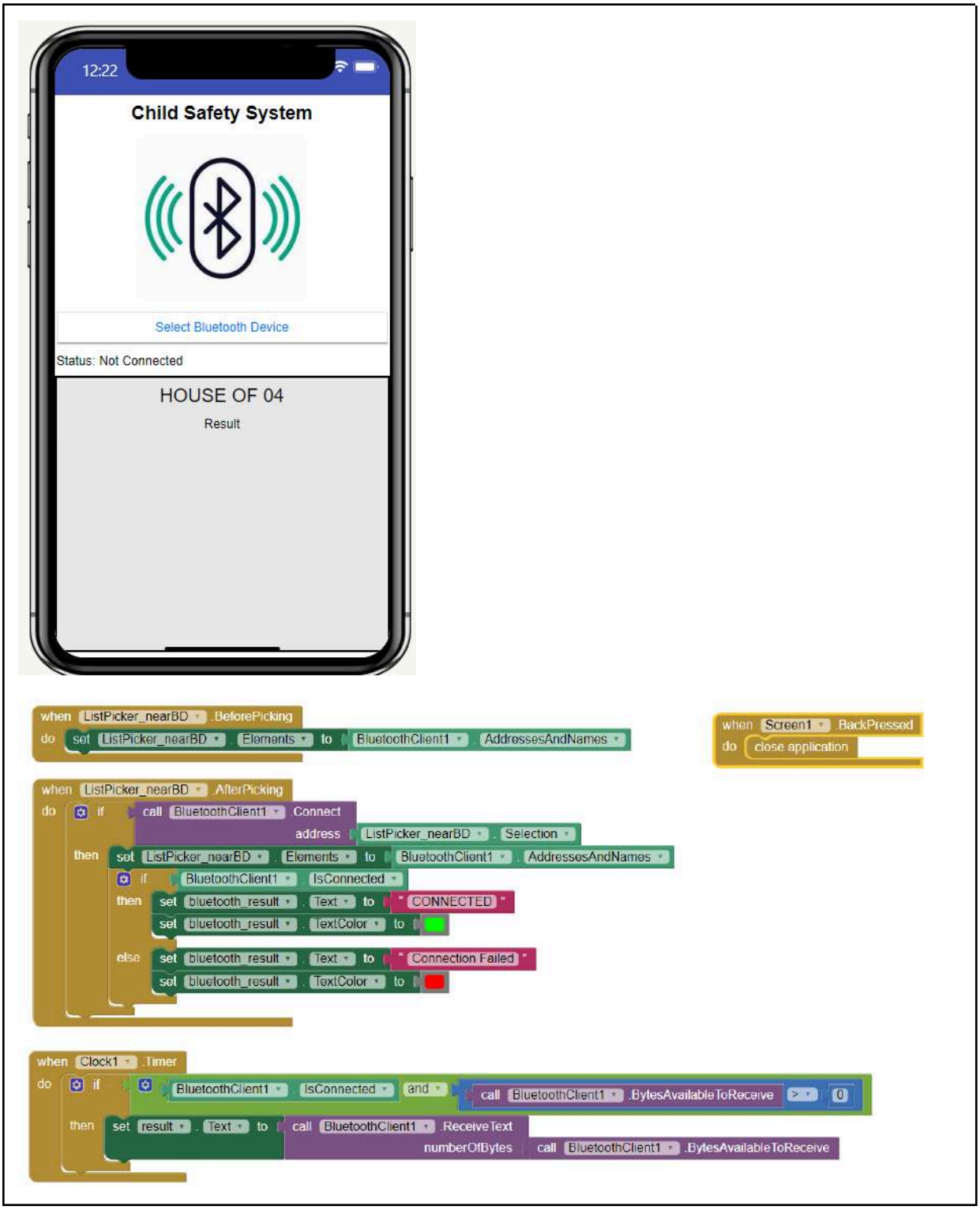


Figure 3.3.3: Bluetooth Connection Interface on MIT App Inventor

The Bluetooth page in the MIT App Inventor application serves as the interface for connecting the smartphone to the STM32F446RE microcontroller via a Bluetooth module (HC-05). This connection allows the app to receive real-time alerts and status updates from the fall prevention system. When the user opens the Bluetooth page, they click on the Select Bluetooth Device button. This action triggers the ListPicker to scan for available Bluetooth devices within range. The ListPicker displays the names of the devices, allowing the user to select the appropriate Bluetooth module (HC-05) connected to the STM32F446RE microcontroller. After the ListPicker displays the available devices, the user selects the HC-05 module. This selection triggers the AfterPicking event of the ListPicker, which prepares the app to connect to the selected device. Once the connection process is initiated, the app continuously updates the Status based on the connection state. If the connection is successful, the label changes to "Connected". If the connection fails, it displays "Connection Failed" or an error message to inform the user. After a successful connection, the app listens for incoming data from the Bluetooth module. This data includes real-time alerts and status updates from the children's fall prevention system. The received messages are displayed on the screen, providing the user with immediate information about any detected fall risks or other relevant alerts. By implementing these blocks and following this workflow, the Bluetooth page in the MIT App Inventor app effectively facilitates the connection between the smartphone and the STM32F446RE microcontroller, ensuring that caregivers receive real-time alerts and updates about potential fall risks, thereby enhancing child safety in high-rise buildings.

CHAPTER 4

RESULT AND DISCUSSION

The result of the project will be included in this chapter. There will be discussion about how the project works. Have the objectives been achieved? Are the real project runs the same as the simulation? All of that will be discussed in this chapter. About the problem that occurs during running the project and the solution should be included also. There was an explanation of how the hardware runs smoothly or not. Discussion in this project is to find a way to solve the problem statement and achieve in this project.

4.1 Prototype

This part describes how the prototype and hardware were created and attached together.

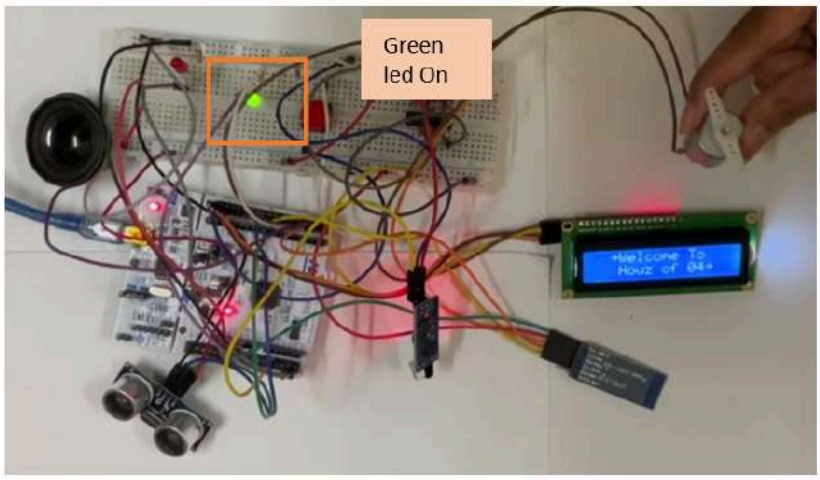


Figure 4.1: circuit on breadboard, nothing detected around the area

Figure 4.1 shows the Children Falling Prevention System in high-risk buildings when there is nothing detected around the area on the breadboard. When the ultrasonic sensor, IR sensor and push button did not pressed, only green LED which indicate the availability turn on, meanwhile red led, buzzer and dc motor off.

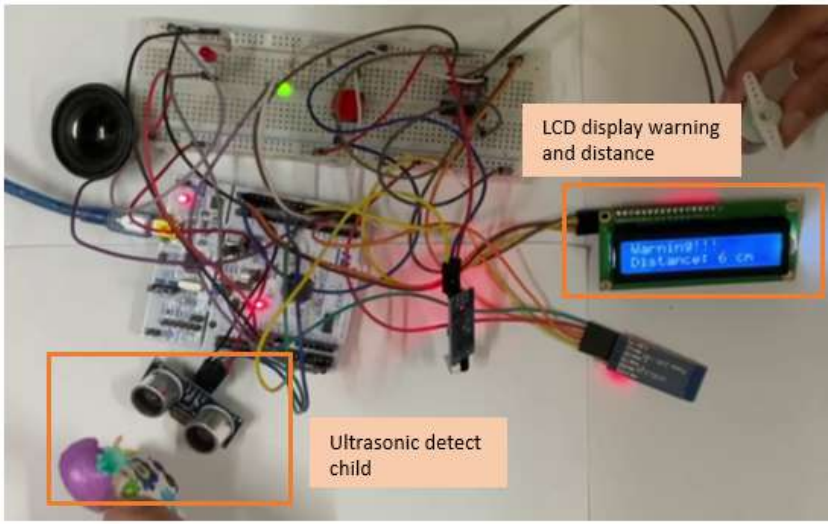


Figure 4.2: ultrasonic sensor detected a child around the area

Figure 4.2 shows circuit on breadboard when ultrasonic sensor detected a child around the area, the LCD displayed “Warning!!” and the distance, the sensor can detect movement up to 15 cm in project, only green LED which indicate the availability turn on, meanwhile red led, buzzer and dc motor off. Since in control did not reach the danger zone yet.

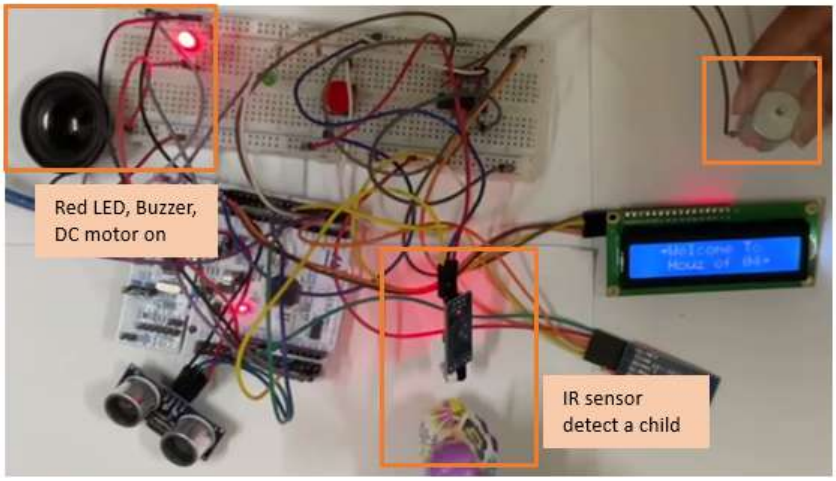


Figure 4.3: IR sensor detected a child around the area

Figure 4.3 shows circuit on breadboard when IR sensor detected a child around the area, IR sensor acted as alarm sensor by means when sensor detected any motion at line area the buzzer and red led turned on, meanwhile the dc motor will close the gates immediately. Only green led off at this moment.

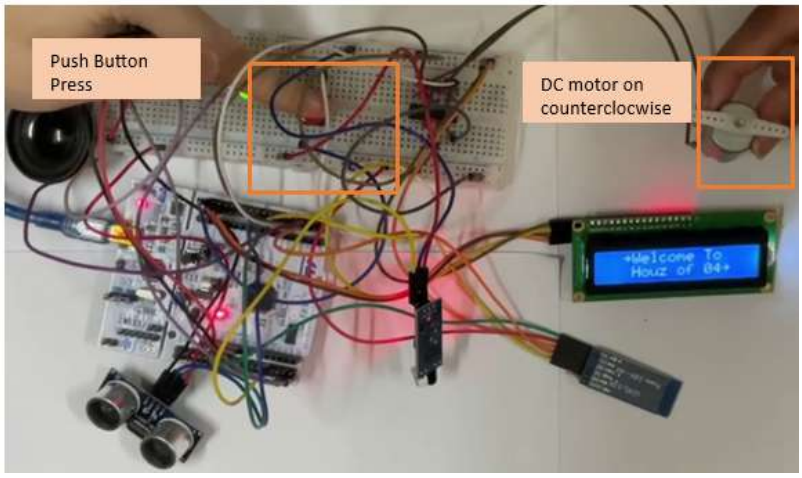


Figure 4.4: when push button pressed

Figure 4.4 shows circuit on breadboard when push button pressed, when push button pressed the dc motor will open the balcony gates. The DC motor will open the gates only when the push button is pressed. At this moment, only green LED which indicates the availability turn on, meanwhile red led, buzzer and dc motor off.

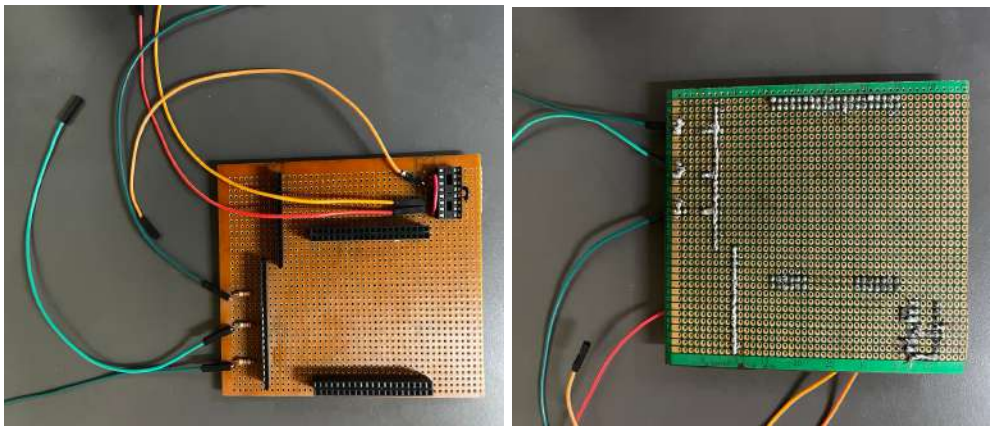


Figure 4.5: donut board full soldered

Figure 4.5 shows the donut board fully soldered. The project, initially designed using an Arduino Uno due to the unavailability of the STM32F466RE in Tinkercad, encountered issues after being transferred to a soldered donut board. The connections, which included components such as LEDs, a buzzer, a push button, an ultrasonic sensor, a Bluetooth module, a DC motor with its driver, and an LCD I2C module, were carefully soldered to replicate the breadboard setup. However, the circuit did not function as expected, likely due to unintended connections or solder bridges between adjacent pads on the donut board. This caused interference and miscommunication among the components, preventing the system from operating correctly despite proper initial prototyping on the breadboard.

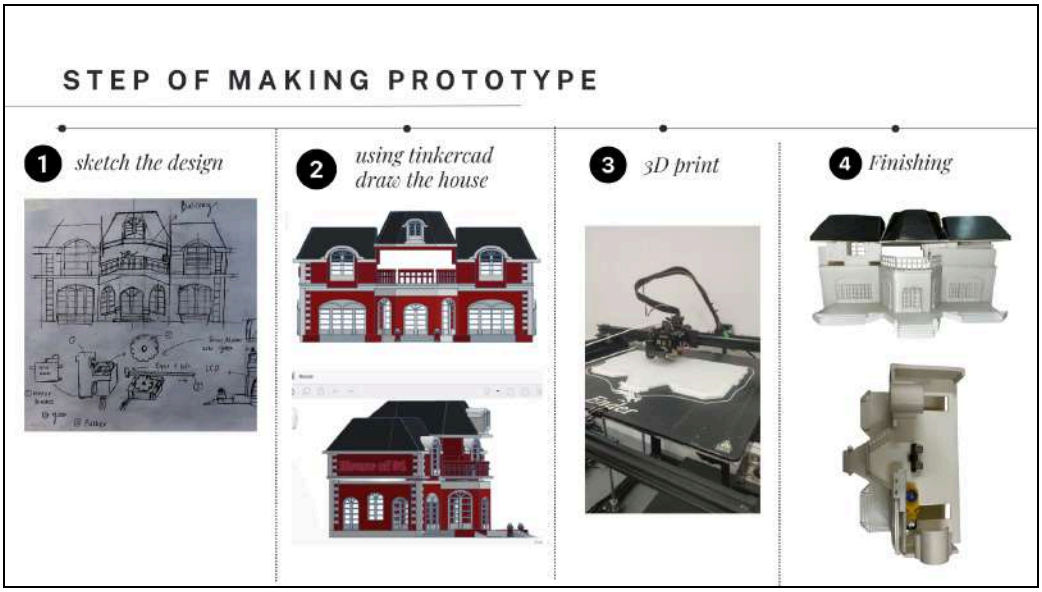


Figure 4.6: Flow of making prototype

Figure 4.6 shows the steps of making a prototype, first, the design was sketched on paper, showing the front and side views of the house, as well as the gear for the motor in its actual size. Then, using Tinkercad, a detailed 3D design was created, here all the detail needs to focus on. Next, the design was 3D printed, ensuring all settings and machine parameters were correctly configured to avoid errors and the need for reprinting. Finally, in the finishing stage, all parts of the house were glued and assembled. The house was created in separate parts to facilitate easier redesigns and put all the hardware components.

4.2 Prototype Operation

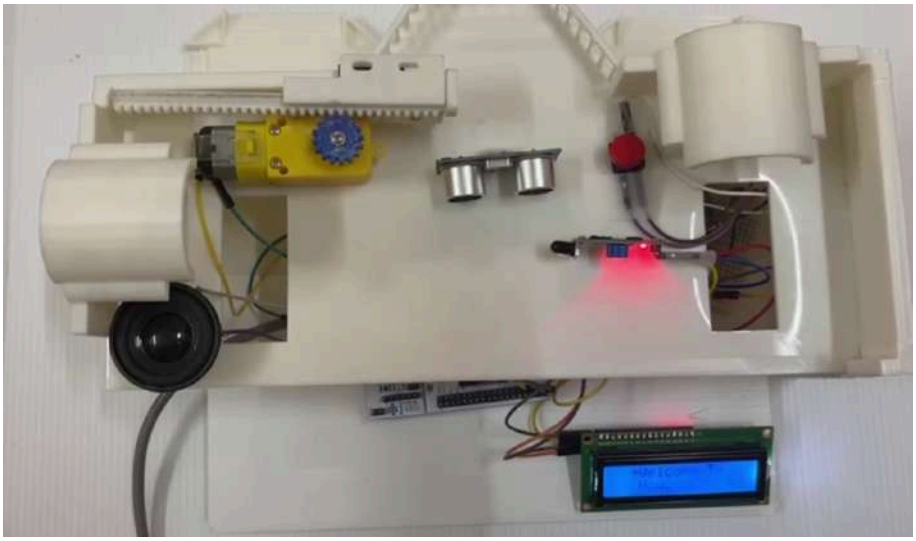


Figure 4.7: on prototype, nothing detected around the area

Figure 4.7 shows the Children Falling Prevention System in high-risk buildings when there is nothing detected around the area on the prototype. When ultrasonic sensor, IR sensor and push button did not pressed, only green LED which indicate the availability turn on, meanwhile red led, buzzer and dc motor off

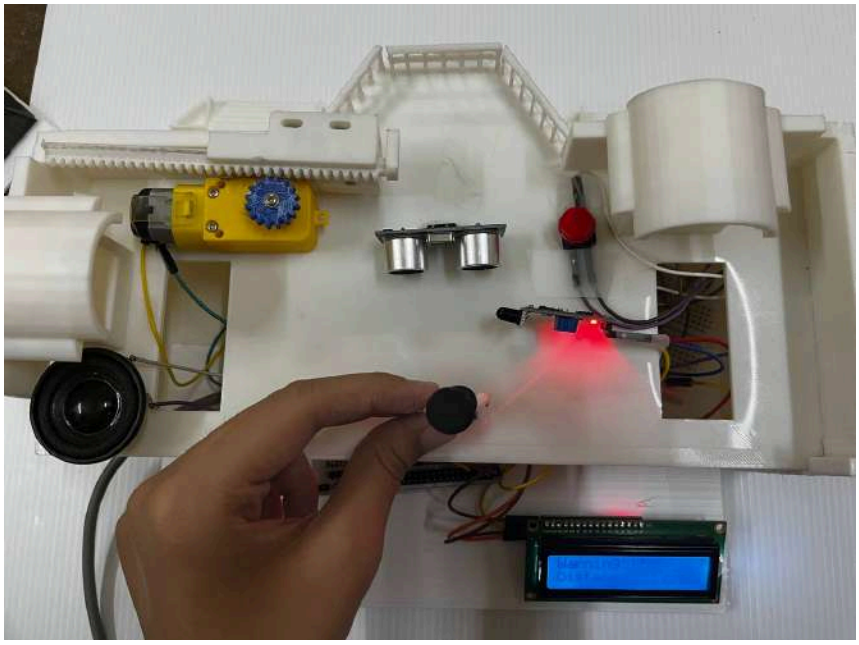


Figure 4.8: On Prototype, ultrasonic sensor detected a child around the area

Figure 4.8 shows the circuit on the prototype. When the ultrasonic sensor detected a child around the area, the LCD displayed “Warning!!” and the distance, the sensor can detect movement up to 15 cm in project, only green LED which indicate the availability turn on, meanwhile red led, buzzer and dc motor off. Since in control did not reach the danger zone yet.

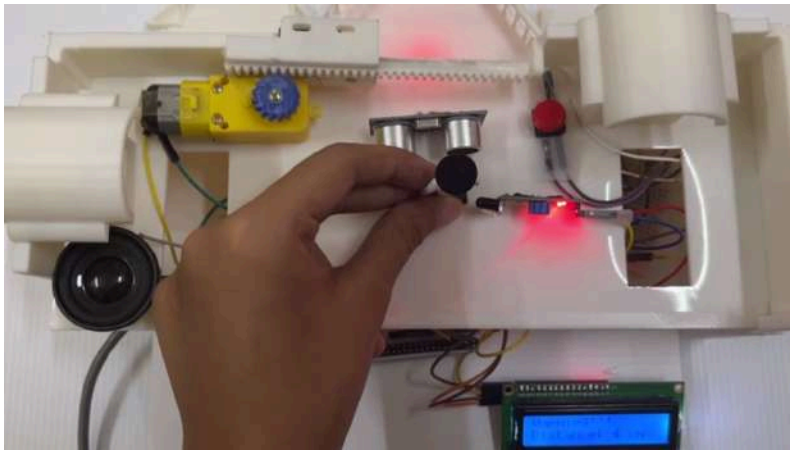


Figure 4.9: IR sensor detected a child around the area

Figure 4.9 shows circuit on prototype when IR sensor detected a child around the area, IR sensor acted as alarm sensor by means when sensor detected any motion at line area the buzzer and red led turned on, meanwhile the dc motor closed the gates immediately. Only green led off at this moment.

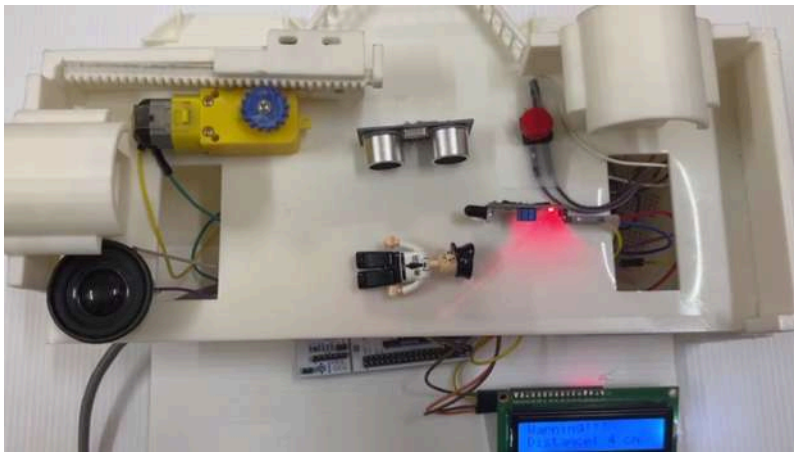


Figure 4.9.1: when push button pressed

Figure 4.9.1 shows circuit on prototype when push button pressed, when push button pressed the dc motor will open the balcony gates. The DC motor will open the gates only when the push button is pressed. At this moment, only green LED which indicates the availability turn on, meanwhile red led, buzzer and dc motor off.

4.3 Test and troubleshoot

In the initial design of the children fall prevention system, a servo motor was used to control the door mechanism. However, during testing, it was observed that the system did not function correctly when connected to the Bluetooth module. The servo motor, which requires precise positioning and consumes relatively high current, likely caused interference or power supply issues that disrupted the Bluetooth communication. Additionally, servo motors are sensitive to power fluctuations, which can occur when other components are drawing power simultaneously. To address these issues, the servo motor was replaced with a DC motor. DC motors are generally simpler to control and less sensitive to power supply variations, making them more compatible with the other components in the circuit. This change ensures more reliable operation of the system, particularly the critical communication between the microcontroller and the smartphone via Bluetooth, thus enhancing overall system stability and effectiveness. The decision to switch from using a servo motor to a DC motor in the children fall prevention system was driven by practical considerations of interference, power supply stability, control complexity, reliability, compatibility, and overall system responsiveness. By opting for DC motors, the system achieves better performance and reliability, ensuring that the critical safety functions are not compromised by the complexities and sensitivities associated with servo motors. This change ultimately enhances the system's ability to protect children from fall risks in high-rise buildings, fulfilling the project's primary objective of creating a robust and effective safety solution.

Then, this project, which was originally built using an Arduino Uno due to the STM32F466RE's unavailability in Tinkercad, ran into some problems when it was transferred to a soldered donut board. The connections, which included LEDs, a buzzer, a push button, an ultrasonic sensor, a Bluetooth module, a DC motor with driver, and an LCD I2C module, were meticulously connected to duplicate the breadboard configuration. However, the circuit did not work as expected, most likely because of unintentional connections or solder bridges between

adjacent pads on the donut board. This resulted in interference and miscommunication among the components, preventing the system from functioning properly despite appropriate breadboard development.

Moreover, in the development of the children fall prevention system, there was a significant challenge with the Bluetooth module's connectivity and data transmission to the MIT App Inventor interface. Despite successfully establishing a Bluetooth connection between the STM32F446RE microcontroller and the MIT App Inventor, the expected output data from the sensors and actuators did not display on the app's screen. This issue persisted even when using alternative applications like Serial Bluetooth Terminal, indicating a deeper underlying problem. MIT App Inventor, while versatile, may have limitations or bugs related to handling continuous Bluetooth data streams, especially when the data includes mixed sensor outputs. Similarly, Serial Bluetooth Terminal, although robust, might require specific configurations to handle data from the STM32F446RE microcontroller. Compatibility issues between the Bluetooth module's firmware and the receiving application's software can also cause data display problems. Given these challenges, the Serial Monitor has been chosen for displaying the output data. The Serial Monitor provides a direct and reliable interface for debugging and verifying data transmission from the STM32F446RE microcontroller. By using the Serial Monitor, it can ensure that the data from the sensors and actuators is accurately received and displayed, providing a clear and reliable means of monitoring the system's performance. This approach allows us to focus on refining the hardware and software functionalities, ensuring the safety and effectiveness of the children fall prevention system in high-rise buildings.

DISCUSSION

The chosen project for this mini project was to develop a Children Falling Prevention System that relied on STM32F466RE Microcontroller. The main function of the system is that the owner can monitor their children, get notification when the child stands closer to the balcony, and also there's an alarm to notify the user. In addition, the balcony system innovation can possibly close the gates automatically when the sensor detects movement around the area. Therefore, to make it user friendly, the system has two light indicators, Red led will turn on and buzzer on when it detects something moving around the area. Otherwise, when it's in normal condition the green LED turns on.

The system integrates various sensors, including ultrasonic and IR sensors, to detect potential fall hazards and movements. The ultrasonic sensor measures the distance from the sensor to the nearest object, helping to identify when a child is dangerously close to an edge. The IR sensor detects movement, triggering an immediate response to close a door or window via a motor, and turning on an alert system. The use of the STM32F446RE microcontroller provided a robust platform for handling the data processing and control logic necessary for these sensors and actuators.

One of the most difficult issues during development was creating a stable Bluetooth connection between the STM32F446RE microcontroller and the MIT App Inventor interface. Despite the successful connection, data transmission to the app did not work as planned, resulting in no output on the screen. This difficulty remained even when using alternate apps, such as Serial Bluetooth Terminal, indicating potential issues with data processing, uart rate synchronisation, or software compatibility. Given the limitations associated with Bluetooth transmission, the team has chosen to show the output data using the Serial Monitor. This decision enabled the team to maintain data integrity and simplify troubleshooting. The Serial Monitor offered quick and reliable input, which was essential to ensuring that the system's components functioned properly. This stage was critical in analysing the system's behaviour and making the necessary changes to increase performance.

Initially, the system utilised a servo motor for sliding door operation. However, due to the circuit not functioning correctly when connected to the Bluetooth module, the team decided to switch to using a DC motor. The DC motor provided more reliable operation under the given

circumstances, ensuring that the sliding door would close securely when a potential fall hazard was detected. This change highlighted the importance of selecting appropriate hardware components that can operate reliably within the system's restrictions.

Aside from the technical issues, the project emphasises the necessity for policy and legislation to enforce the implementation of safety precautions for children in high-rise buildings. While the system itself can considerably improve safety, its success is dependent on widespread adoption and attention to safety requirements. This initiative pushes for the incorporation of child safety designs into new and existing high-rise houses, with regulatory frameworks to ensure compliance.

For Simulation part, the circuit of this project has been constructed using the TinkerCad Software which is available on the Web. TinkerCad is a software that allows users to Construct 3D Design Objects, Codeblocks and more. As for this project, used the TinkerCad software to construct and simulate the Child Falling Preventer Circuit System in real time. The reason why the simulation construct used TinkerCad is to check the connection between the microcontroller and all the components. Even though, in TinkerCad the microcontroller that is used is Arduino Uno and also the coding needs to be used with Arduino pins. Moreover, the problem that I faced is the LCD I2C is not working in TinkerCad, therefore, the connection needed to run on the hardware part, also by using Arduino Uno.

For the hardware part, the initial connections were made using an Arduino Uno to prevent the STM32F466RE from being damaged due to incorrect connections, which could cause a short circuit or other issues that might harm the board. After the setup successfully met the target, the microcontroller was switched to the STM32F466RE, and the code for initialising the pins was updated accordingly. The problem encountered while running the code was that when the Bluetooth module code was included, the servo motor would not operate. This issue was resolved by replacing the servo motor with a DC motor. The functions of the servo motor and the DC motor remain the same, which is to open and close the balcony gates to prevent the child from going toward the balcony.

For the prototype part, the design process started with sketches, including the house and all the components for the balcony system. Next, the entire 3D design was created using TinkerCad, which required careful attention. After finalising the design, by using an ultimaker

cura application, to slice the design and save the file before print, also to ensure that all machine parameters and settings were correctly configured to prevent errors and the need for reprinting. During the finishing phase, all of the house's components were assembled and glued together. To facilitate easier redesigns and incorporate all the hardware components, the house was designed in sections. The problem encountered was that the design needed to be printed in three parts which are the first floor, the second floor, and the roof. After printing, some parts of the second floor were missing, making it difficult to attach to the roof, and the actual size was smaller than intended. This issue was resolved by making the roof easily removable to allow access to the components inside the house.

Future adjustments of this project could look into a variety of enhancements. Improved Bluetooth connection, obtained through greater analysis of current difficulties, could result in more dependable data transmission and improved interaction with mobile applications. Sensor accuracy could be improved by using more advanced sensors, allowing the system to detect fall hazards and movement with higher precision. Creating a more straightforward and user-friendly interface for guardians would make the system easier to monitor and manage. Furthermore, working with legislators to promote the adoption of obligatory kid safety features in high-rise buildings could lead to wider implementation and improved overall safety.

Finally, this project has demonstrated the potential of developing a fall prevention system for children in high-rise buildings using the STM32F446RE microcontroller. While problems such as Bluetooth connectivity issues were encountered, the successful execution of essential safety measures and the switch to using the Serial Monitor for data display demonstrate the system's capabilities. With more research and regulatory backing, this technology can play an important role in reducing accidents and assuring the safety of children in high-rise residential environments.

CHAPTER 5

CONCLUSION

This chapter will highlight the contribution of findings to new knowledge applications. It also discusses the limitations of the component that is used in carrying out the project development and the scope for further development based on the results.

5.1 Conclusion

In conclusion, a hectic lifestyle can cause parents to overlook their children, potentially leading to unpleasant incidents at the house balcony. However, technological advancements will undoubtedly make the task easier. By analysing people's experiences with their open space balcony, there is news about children falling from house balconies while their parents do the household or cooking, therefore “Children Falling Prevention System in high-rise buildings” was developed.

With the specially selected sensor, which is dependable and economical, the development cost was reduced by more than 50%, making the entire system affordable to the public. The following were the objectives that were accomplished:

1. To design software and hardware for a Children Falling Prevention System in high-rise buildings using the STM32F466RE microcontroller.

   This is accomplished by developing custom software to monitor and detect potential fall risks, using sensors and hardware components to effectively implement the detection system, and programming the STM32F466RE microcontroller using the Arduino IDE c++ coding for real-time processing and response. The system includes an LCD display that shows a warning when children are detected near the balcony, triggering an immediate alarm and

automatically closing the balcony gates. Additionally, notifications are sent to the ‘Houz’ design application. The gates can only be opened by pressing a push button.

2. To build a prototype of the Children Falling Prevention System for high-rise buildings.

This is accomplished by investigating 3D design using TinkerCad and utilising Ultimaker's Cura for printing. All components are soldered and placed in the prototype. Users can browse their phones with peace of mind, as they will receive notifications from the 'Houz' application when their child is near the balcony.

3. To test and troubleshoot the prototype of project Children Falling Prevention system.

This is successfully achieved by simply placing the produced prototype through a variety of tests. The prototype and the component is perfectly placed and the system can be seen clearly.

Understanding the functions of other components allows the circuit simulation to proceed as it provides the basis for achieving the desired result. The component covered in Chapter 2 is chosen as a result of the information gathering procedure. It will be easier to simulate the circuit after completing some study on the components.

In summary, based on the results, this project effectively prevents children from falling in high-rise buildings. It also helps users feel more at ease at home, as all alarms, sensors, buttons, LCD displays, and notifications immediately alert them if something is detected around the balcony area.

The completion of this project successfully demonstrated the development of a children's fall prevention system for high-rise buildings using the STM32F446RE microcontroller. By

integrating many sensors and actuators, the system effectively detects possible fall dangers and informs loved ones, improving kid safety in urban contexts. Despite meeting difficulties with Bluetooth connection and motor control, the project indicated critical areas for improvement, such as the need for more dependable data transmission, improved sensor accuracy, and more appealing interface. Future improvements and policy representation may strengthen this system's function in preventing child falls and providing better living conditions in high-rise buildings. Overall, the project highlights the significance of using current technology to address crucial safety concerns, and it provides a potential approach for protecting vulnerable populations in highly populated metropolitan areas.

REFERENCES

Here are the citations formatted in IEEE style:

[1] Ultimaker, "How to use a 3D printer," accessed July 7, 2024. [Online]. Available: https://ultimaker.com/learn/how-to-use-a-3d-printer/

[2] L. Carolo, D. Pechter, and B. Visser, "3D printed gears: Get the gear that fits your needs," All3DP, December 15, 2023. [Online]. Available: https://all3dp.com/2/3d-printed-gears-get-the-gear-that-fits-your-needs/ . Accessed: July 7, 2024.

[3] Centers for Disease Control and Prevention, "Preventing falls and hip fractures," May 9, 2024. [Online]. Available: https://www.cdc.gov/falls/hip-fractures.html . Accessed: July 9, 2024.

[4] New York State Department of Health, "Childhood and teen fall prevention," October 2022. [Online]. Available: https://www.health.ny.gov/prevention/injury_prevention/children/falls.htm . Accessed: July 9, 2024.

[5] Rhode Island Department of Health, "Childhood falls," accessed July 9, 2024. [Online]. Available: https://health.ri.gov/healthrisks/injury/falls/

[6] "Servomotor and Bluetooth module block each other," Stack Exchange, May 5, 2020. [Online]. Available: https://arduino.stackexchange.com/questions/80124/servomotor-and-bluetooth-module-block-each-other. Accessed: July 11, 2024.